

university of groningen

Models and Semantics of Computation (MaSC)

Lecture 13: Bisimilarity and types in the π-calculus

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands

October 22, 2026

The π-calculus



Last time:

- The π-calculus: extension of CCS with dynamic communication topologies.
- Key technical ingredients: name passing and scope extrusion.
- Two approaches to operational semantics:
  reduction semantics (closed systems) and LTS (open systems).

Today: Strong bisimilarity and a linear type system (Kobayashi et al.)

The π-calculus: Syntax



π	$::= \overline{x}\, y$	send y on x
	$\mid x(y)$	receive on x and store in y
	$\mid \tau$	internal action
p, q	$::= \mathrm{nil}$	inactive process
	$\mid \pi.p$	action prefix
	$\mid [x = y]p$	matching (think: if $x = y$ then p)
	$\mid p + q$	non deterministic choice
	$\mid p \mid q$	parallel composition
	$\mid (y)p$	restriction (similar to $p \backslash y$, name y is local to p)
	$\mid !p$	replication (think: $p \mid p \mid \cdots$)



Strong Bisimilarity

Early Ground Bisimulation

Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong early ground bisimulation:

$$p \mathcal{R} q \Rightarrow \begin{cases} p \xrightarrow{\tau} p' & \Rightarrow \exists q'.\ q \xrightarrow{\tau} q' \wedge p' \mathcal{R} q' \end{cases}$$

Early Ground Bisimulation

Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong early ground bisimulation:

$$p \mathcal{R} q \Rightarrow \begin{cases} p \xrightarrow{\tau} p' & \Rightarrow \exists q'.\ q \xrightarrow{\tau} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}\, y} p' & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}\, y} q' \wedge p' \mathcal{R} q' \end{cases}$$

Early Ground Bisimulation

Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong early ground bisimulation:

$$p \mathcal{R} q \Rightarrow \begin{cases} p \xrightarrow{\tau} p' & \Rightarrow \exists q'.\ q \xrightarrow{\tau} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}\, y} p' & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}\, y} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}(y)} p' \ (y \notin \mathsf{fn}(q)) & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}(y)} q' \wedge p' \mathcal{R} q' \end{cases}$$

Early Ground Bisimulation

Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong early ground bisimulation:

$$p \mathcal{R} q \Rightarrow \begin{cases} p \xrightarrow{\tau} p' & \Rightarrow \exists q'.\ q \xrightarrow{\tau} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}\, y} p' & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}\, y} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}(y)} p' \ (y \notin \mathrm{fn}(q)) & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}(y)} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{x(y)} p' \ (y \notin \mathrm{fn}(q)) & \Rightarrow \forall z\, . \exists q'.\ q \xrightarrow{x(y)} q' \wedge p'[z/y] \mathcal{R} q'[z/y] \end{cases}$$

Early Ground Bisimulation



Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong early ground bisimulation:

$$p \mathcal{R} q \Rightarrow \begin{cases} p \xrightarrow{\tau} p' & \Rightarrow \exists q'.\ q \xrightarrow{\tau} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}\, y} p' & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}\, y} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{\overline{x}(y)} p' \ (y \notin \mathrm{fn}(q)) & \Rightarrow \exists q'.\ q \xrightarrow{\overline{x}(y)} q' \wedge p' \mathcal{R} q' \\ p \xrightarrow{x(y)} p' \ (y \notin \mathrm{fn}(q)) & \Rightarrow \forall z\ . \exists q'.\ q \xrightarrow{x(y)} q' \wedge p'[z/y] \mathcal{R} q'[z/y] \\ \text{(and vice-versa)} \end{cases}$$

- In bound output and input, the condition '$y \notin \mathrm{fn}(q)$' allows to consider only moves where y is fresh for both p and q.
- In bound output, also names z which are not fresh (i.e. appear free in p' and q') can replace variable y.
- It is **early** because q can choose to move depending on the received value z.

Early Ground Bisimulation, Concisely



Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong early ground bisimulation:

$$p\ \mathcal{R}\ q \Rightarrow \begin{cases} p \xrightarrow{\alpha} p' & \text{with } \alpha \neq x(y) \wedge \mathsf{bn}(\alpha) \cap \mathsf{fn}(q) = \emptyset \\ & \Rightarrow \exists q'.\ q \xrightarrow{\alpha} q' \wedge p'\ \mathcal{R}\ q' \\ p \xrightarrow{x(y)} p'\ (y \notin \mathsf{fn}(q)) & \Rightarrow \forall z\ .\ \exists q'.\ q \xrightarrow{x(y)} q' \wedge p'[z/y]\ \mathcal{R}\ q'[z/y] \\ \text{(and vice-versa)} & \end{cases}$$

$p \sim_{\mathtt{E}} q$ iff exists a strong early ground bisimulation $\mathcal{R}$ with $(p, q) \in \mathcal{R}$

Late Ground Bisimulation



Let $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$. We define a strong **late** ground bisimulation:

$$p\ \mathcal{R}\ q \Rightarrow \begin{cases} p \xrightarrow{\alpha} p' & \text{with } \alpha \neq x(y) \wedge \mathrm{bn}(\alpha) \cap \mathrm{fn}(q) = \emptyset \\ & \Rightarrow \exists q'.\ q \xrightarrow{\alpha} q' \wedge p'\ \mathcal{R}\ q' \\ p \xrightarrow{x(y)} p' \ (y \notin \mathrm{fn}(q)) & \Rightarrow \exists q'.\ q \xrightarrow{x(y)} q' \wedge \forall z\,.\, p'[z/y]\ \mathcal{R}\ q'[z/y] \\ \text{(and vice-versa)} & \end{cases}$$

- We require a single input transition of q to q' such that p' and q' are related for any received value.
- That is, q must choose a move without knowing what the input value will be.

$p \sim_{\mathrm{L}} q$ iff exists a strong late ground bisimulation $\mathcal{R}$ with $(p, q) \in \mathcal{R}$

Early vs Late

Early: For every input **Bob** can choose a different move

$$\forall z.\, \exists q'.\, \ldots$$

Late: **Bob** must choose one move, valid for every input

$$\exists q'.\, \forall z.\, \ldots$$

Early vs Late: Example



Consider $P \triangleq x(y).\tau + x(y)$ and $Q \triangleq P + x(y).[y = a]\tau$.

- $P \xrightarrow{x(z)} \tau \xrightarrow{\tau} \mathrm{nil}$ and $P \xrightarrow{x(z)} \mathrm{nil}$ (with z fresh)
- $Q \xrightarrow{x(z)} \tau \xrightarrow{\tau} \mathrm{nil}$ and $Q \xrightarrow{x(z)} \mathrm{nil}$ and $Q \xrightarrow{x(z)} [z = a]\tau$ (with z fresh)

Early game:

- Alice picks $Q \xrightarrow{x(z)} [z = a]\tau$
- For $z = a$, Bob picks $P \xrightarrow{x(z)} \tau$
- For $z \neq a$, Bob picks $P \xrightarrow{x(z)} \mathrm{nil}$
- $\rightarrow$ Bob wins and $p \sim_{\mathtt{E}} q$.

Late game:

- Alice picks $Q \xrightarrow{x(z)} [z = a]\tau$
- If Bob picks $P \xrightarrow{x(z)} \tau$, Alice wins when $z \neq a$.
- If Bob picks $P \xrightarrow{x(z)} \mathrm{nil}$, Alice wins when $z = a$
- $\rightarrow$ Alice wins and $p \not\sim_{\mathtt{L}} q$.

Early vs Late: Properties



- $P \sim_{\mathtt{L}} Q \Rightarrow P \sim_{\mathtt{E}} Q$ but $P \sim_{\mathtt{E}} Q \not\Rightarrow P \sim_{\mathtt{L}} Q$ (previous example).
- $\sim_{\mathtt{E}}$ and $\sim_{\mathtt{L}}$ are equivalences but not congruences:
  they are not preserved by input prefix.
- Define

$$p \simeq_{\mathtt{E}} q \quad \text{iff } p\sigma \sim_{\mathtt{E}} q\sigma \text{ for every substitution } \sigma$$
$$p \simeq_{\mathtt{L}} q \quad \text{iff } p\sigma \sim_{\mathtt{L}} q\sigma \text{ for every substitution } \sigma$$

- $\simeq_{\mathtt{E}}$ and $\simeq_{\mathtt{L}}$ are congruences:
  the largest congruences included in each bisimilarity.



A Linear Type System

Linearity and the π-calculus



- Paper by Naoki Kobayashi, Benjamin C. Pierce and David N. Turner.
- First appeared in POPL, the annual Symposium on Principles of Programming Languages.
- Journal version in TOPLAS (Transactions on Programming Languages and Systems).
- Very influential paper: elegant set-up and powerful reasoning techniques for processes.

Paper Contents



1. **Introduction:** From arity types to polarities and linear types.
2. **Notational preliminaries:** Syntax of processes and types, type environments, operations on types.

Paper Contents



1. **Introduction:** From arity types to polarities and linear types.
2. **Notational preliminaries:** Syntax of processes and types, type environments, operations on types.
3. **Typing:** Typing rules and lemmas (weakening, strengthening, substitution).
4. **Operational semantics:**
   - Structural congruence and guarded processes (and their properties, Lem 4.1.1).
   - Labelled reduction relation for annotated processes (and its properties).
   - Soundness (Thm 4.3.1), correspondence with untyped semantics (Thm 4.3.2).
   - Partial confluence (Thm 4.4.3).

Paper Contents



1. **Introduction:** From arity types to polarities and linear types.
2. **Notational preliminaries:** Syntax of processes and types, type environments, operations on types.
3. **Typing:** Typing rules and lemmas (weakening, strengthening, substitution).
4. **Operational semantics:**
   - Structural congruence and guarded processes (and their properties, Lem 4.1.1).
   - Labelled reduction relation for annotated processes (and its properties).
   - Soundness (Thm 4.3.1), correspondence with untyped semantics (Thm 4.3.2).
   - Partial confluence (Thm 4.4.3).
5. **Bisimulation:** Definitions and main result (congruence, Thm 5.25).
6. **Examples:** Two different implementations and proof of their equivalence.
7. **Applications:** Implementing channels, linear channels, replicated channels.

Paper Contents



1. **Introduction:** From arity types to polarities and linear types.
2. **Notational preliminaries:** Syntax of processes and types, type environments, operations on types.
3. **Typing:** Typing rules and lemmas (weakening, strengthening, substitution).
4. **Operational semantics:**
   - Structural congruence and guarded processes (and their properties, Lem 4.1.1).
   - Labelled reduction relation for annotated processes (and its properties).
   - Soundness (Thm 4.3.1), correspondence with untyped semantics (Thm 4.3.2).
   - Partial confluence (Thm 4.4.3).
5. **Bisimulation:** Definitions and main result (congruence, Thm 5.25).
6. **Examples:** Two different implementations and proof of their equivalence.
7. **Applications:** Implementing channels, linear channels, replicated channels.

In this presentation: I will focus on Sections 1-4.

Syntax of Processes



$P, Q ::= P \mid Q$	parallel composition
$\mid \overline{x}\langle \tilde{y} \rangle$	output atom
$\mid x(\tilde{y}).P$	input prefix
$\mid\, !x(\tilde{y}).P$	replicated input
$\mid (y : T)P$	restriction (with type annotation)
$\mid \text{if } x \text{ then } P \text{ else } Q$	conditional

Some differences:

- Polyadic communication ($\tilde{y}$ denotes a finite sequence of names).
- No continuation after outputs (only 'atoms').
- No inactive process, mismatching, nor non-deterministic choice.

Syntax of Types



We specify name behavior by specifying:

- Multiplicities, denoted m, can be 1 or ω (linear = exactly once and unlimited).
- Polarities, denoted p, q, are subsets of $\{i, o\}$ (input and output capabilities).

Useful abbreviations:

- $\updownarrow = \{i, o\}$, $\emptyset = \{\}$ (even polarities)
- $! = \{o\}$
- $? = \{i\}$

Linear Types



Multiplicities and polarities are used to define the set of types:

$$
\begin{array}{lll}
T ::= \mathsf{p}^{m}[\tilde{T}] & & \text{channel types} \\
\quad \mid \mathsf{bool} & & \text{booleans}
\end{array}
$$

(In examples, we use a base type num for numbers).

Example

Linear Types



Multiplicities and polarities are used to define the set of types:

$$T ::= \mathsf{p}^{m}[\tilde{T}] \qquad \text{channel types}$$
$$\mid \mathsf{bool} \qquad \text{booleans}$$

(In examples, we use a base type num for numbers).

Example

- $!^{1}[\mathsf{bool}]$: a channel that can be used exactly once to output a boolean.

Linear Types



Multiplicities and polarities are used to define the set of types:

$$T ::= \mathsf{p}^{m}[\tilde{T}] \qquad \text{channel types}$$
$$\mid \mathsf{bool} \qquad \text{booleans}$$

(In examples, we use a base type num for numbers).

Example

- $!^{1}[\mathsf{bool}]$: a channel that can be used exactly once to output a boolean.
- $\updownarrow^{1}[\mathsf{bool}]$: a channel that can be used once for input and once for output.

Linear Types



Multiplicities and polarities are used to define the set of types:

$$
\begin{array}{rll}
T ::= & \mathsf{p}^{m}[\tilde{T}] & \text{channel types} \\
\mid & \mathsf{bool} & \text{booleans}
\end{array}
$$

(In examples, we use a base type num for numbers).

Example

- $!^{1}[\mathsf{bool}]$: a channel that can be used exactly once to output a boolean.
- $\updownarrow^{1}[\mathsf{bool}]$: a channel that can be used once for input and once for output.
- $?^{1}[\mathsf{bool}, !^{1}[\mathsf{bool}]]$: a channel that can be used once to receive two messages: a boolean and a linear channel to output a boolean.

Operations on Types



The **combination** of T_1 and T_2, denoted $T_1 + T_2$, is defined as

$$\begin{aligned} \text{bool} + \text{bool} &= \text{bool} \\ \mathsf{p}^{\omega}[\tilde{T}] + \mathsf{q}^{\omega}[\tilde{T}] &= (\mathsf{p} \cup \mathsf{q})^{\omega}[\tilde{T}] \\ \mathsf{p}^{1}[\tilde{T}] + \mathsf{q}^{1}[\tilde{T}] &= (\mathsf{p} \cup \mathsf{q})^{1}[\tilde{T}] \quad \text{if } \mathsf{p} \cap \mathsf{q} = \emptyset \end{aligned}$$

(The operator is undefined in all other cases.)

Typing Environments (1)



- A **type environment** Γ is a set of bindings of distinct variables to types:

$$x_1 : T_1, \ldots, x_n : T_n \qquad \text{(notation: } \tilde{x} : \tilde{T}\text{)}$$

  Type environments contain at least the bindings `tt` : bool and `ff` : bool.

Typing Environments (1)



- A **type environment** Γ is a set of bindings of distinct variables to types:

$$x_1 : T_1, \ldots, x_n : T_n \qquad \text{(notation: } \tilde{x} : \tilde{T}\text{)}$$

  Type environments contain at least the bindings `tt` : bool and `ff` : bool.
- The domain of Γ, written $\mathrm{dom}(\Gamma)$ is the set of variables it binds.

Typing Environments (1)



- A **type environment** Γ is a set of bindings of distinct variables to types:

$$x_1 : T_1, \ldots, x_n : T_n \qquad \text{(notation: } \tilde{x} : \tilde{T}\text{)}$$

  Type environments contain at least the bindings `tt` : bool and `ff` : bool.
- The domain of Γ, written $\mathrm{dom}(\Gamma)$ is the set of variables it binds.
- We write $\Gamma(x) = T$ whenever $\Gamma = \Gamma_1, x : T$, for some Γ_1.

Typing Environments (2)



- Combination extends to typing environments:

$$(\Gamma_1 + \Gamma_2)(x) = \begin{cases} \Gamma_1(x) + \Gamma_2(x) & \text{if } x \in \mathrm{dom}(\Gamma_1) \cap \mathrm{dom}(\Gamma_2) \\ \Gamma_1(x) & \text{if } x \in \mathrm{dom}(\Gamma_1) \text{ and } x \notin \mathrm{dom}(\Gamma_2) \\ \Gamma_2(x) & \text{if } x \in \mathrm{dom}(\Gamma_2) \text{ and } x \notin \mathrm{dom}(\Gamma_1) \end{cases}$$

Using combination, we can correctly split the linear types in a typing environment.

Typing



Typing Judgments

$$\Gamma \vdash P$$

P is well typed under the typing assumptions in Γ and uses each linear capability in Γ just once.

Typing

Typing Judgments

$$\Gamma \vdash P$$

P is well typed under the typing assumptions in Γ and uses each linear capability in Γ just once.

The typing rules rely on an auxiliary notion:

- Types of the form $\mathsf{p}^1[\tilde{T}]$, with $\mathsf{p} \neq \emptyset$ are limited.
  All other types (multiplicity ω, polarity $\emptyset$) are unlimited.
- Γ is unlimited if, for every $x \in \mathrm{dom}(\Gamma)$, the type $\Gamma(x)$ is unlimited.

Typing Rules: Parallel Composition

$$\frac{\Gamma_1 \vdash P \qquad \Gamma_2 \vdash Q}{\Gamma_1 + \Gamma_2 \vdash P \mid Q}$$

- The typing $P \mid Q$ consumes the sum of the resources used in the separate proofs that P and Q are well typed.
- $\Gamma_1 + \Gamma_2$ must be well defined.

Typing Rules: Output

$$\frac{\Gamma \text{ unlimited}}{\Gamma + x : !^m[\tilde{T}] + \tilde{y} : \tilde{T} \vdash \overline{x}\langle\tilde{y}\rangle}$$

- Outputs are the leaves of the typing derivation.
  To ensure linearity, we check that all types in Γ have either multiplicity ω or polarity $\emptyset$.

Typing Rules: Input

$$\frac{\Gamma, \tilde{z} : \tilde{T} \vdash P}{\Gamma + x : ?^m[\tilde{T}] \vdash x(\tilde{z}).P}$$

- An input process is well typed if its body P is well typed in an environment where the capability to input from x has been removed (if it is linear) and the capabilities associated with $\tilde{z}$ have been added.

Typing Rules: Input

$$\frac{\Gamma, \tilde{z} : \tilde{T} \vdash P}{\Gamma + x : ?^m[\tilde{T}] \vdash x(\tilde{z}).P}$$

$$\frac{\Gamma, \tilde{z} : \tilde{T} \vdash P \qquad \Gamma \text{ unlimited}}{\Gamma + x : ?^\omega[\tilde{T}] \vdash !x(\tilde{z}).P}$$

- An input process is well typed if its body P is well typed in an environment where the capability to input from x has been removed (if it is linear) and the capabilities associated with $\tilde{z}$ have been added.
- For replicated input, the situation is similar.
  The type of x must be unlimited and, since P may be instantiated many times, we must ensure that Γ does not contain linear types.

Typing Rules: Restriction and Conditional



$$\frac{\Gamma, x : \mathrm{p}^m[\tilde{T}] \vdash P}{\Gamma \vdash (x : \mathrm{p}^m[\tilde{T}])P}$$

- The rule for restriction adds $x : \mathrm{p}^m[\tilde{T}]$ to the type environment and checks that P is well typed in the resulting type environment.

Typing Rules: Restriction and Conditional

$$\frac{\Gamma, x : \mathrm{p}^m[\tilde{T}] \vdash P}{\Gamma \vdash (x : \mathrm{p}^m[\tilde{T}])P}$$

$$\frac{\Gamma \vdash P_1 \qquad \Gamma \vdash P_2}{\Gamma + b : \mathsf{bool} \vdash \mathrm{if}\ b\ \mathrm{then}\ P_1\ \mathrm{else}\ P_2}$$

- The rule for restriction adds $x : \mathrm{p}^m[\tilde{T}]$ to the type environment and checks that P is well typed in the resulting type environment.
- For conditionals, we know that P_1 and P_2 will never both be executed, so any linear types in the environment should be used up by both processes.

Example: Enforcing Linearity



Consider the process

$$x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$$

with the following intended channel type for $x : ?^1[!^1[\mathsf{num}], \mathsf{num}]$.

Example: Enforcing Linearity



Consider the process

$$x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$$

with the following intended channel type for $x : ?^1[!^1[\mathsf{num}], \mathsf{num}]$.

- The typing derivation:

$$\frac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

Example: Enforcing Linearity



Consider the process

$$x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$$

with the following intended channel type for $x : ?^1[!^1[\mathsf{num}], \mathsf{num}]$.

- The typing derivation:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

- We need $\Gamma_1 = \Gamma_2 = \{y : !^1[\mathsf{num}], n : \mathsf{num}\}$ with defined $\Gamma_1(y) + \Gamma_2(y)$.

Example: Enforcing Linearity



Consider the process

$$x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$$

with the following intended channel type for $x : ?^1[!^1[\mathsf{num}], \mathsf{num}]$.

- The typing derivation:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

- We need $\Gamma_1 = \Gamma_2 = \{y : !^1[\mathsf{num}], n : \mathsf{num}\}$ with defined $\Gamma_1(y) + \Gamma_2(y)$.
- But this is impossible: $!^1[\mathsf{num}] + !^1[\mathsf{num}]$ is undefined (since $\{!\} \cap \{!\} \neq \emptyset$).

Example: Enforcing Linearity



Consider the process

$$x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$$

with the following intended channel type for $x : ?^1[!^1[\mathsf{num}], \mathsf{num}]$.

- The typing derivation:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

- We need $\Gamma_1 = \Gamma_2 = \{y : !^1[\mathsf{num}], n : \mathsf{num}\}$ with defined $\Gamma_1(y) + \Gamma_2(y)$.
- But this is impossible: $!^1[\mathsf{num}] + !^1[\mathsf{num}]$ is undefined (since $\{!\} \cap \{!\} \neq \emptyset$).
- Hence, $y : !^1[\mathsf{num}]$ can only be in either Γ_1 or Γ_2, but not in both.

Example: Beyond Linearity



Consider again the process $x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$ now with the following intended channel type for $x : ?^1[!^\omega[\mathsf{num}], \mathsf{num}]$.

Example: Beyond Linearity



Consider again the process $x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$ now with the following intended channel type for $x : ?^1[!^\omega[\mathsf{num}], \mathsf{num}]$.

- The typing derivation is similar as before:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

Example: Beyond Linearity



Consider again the process $x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$ now with the following intended channel type for $x : ?^1[!^\omega[\mathsf{num}], \mathsf{num}]$.

- The typing derivation is similar as before:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

- We need $\Gamma_1 = \Gamma_2 = \{y : !^\omega[\mathsf{num}], n : \mathsf{num}\}$ with defined $\Gamma_1(y) + \Gamma_2(y)$.

Example: Beyond Linearity



Consider again the process $x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$ now with the following intended channel type for $x : ?^1[!^\omega[\mathsf{num}], \mathsf{num}]$.

- The typing derivation is similar as before:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y, n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

- We need $\Gamma_1 = \Gamma_2 = \{y : !^\omega[\mathsf{num}], n : \mathsf{num}\}$ with defined $\Gamma_1(y) + \Gamma_2(y)$.
- Now this works, because $!^\omega[\mathsf{num}] + !^\omega[\mathsf{num}]$ is defined.

Example: Beyond Linearity



Consider again the process $x(y,n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)$ now with the following intended channel type for $x : ?^1[!^\omega[\mathsf{num}], \mathsf{num}]$.

- The typing derivation is similar as before:

$$\dfrac{\dfrac{\Gamma_1 \vdash \overline{y}\langle n\rangle \qquad \Gamma_2 \vdash \overline{y}\langle n\rangle}{y : !^1[\mathsf{num}], n : \mathsf{num} \vdash \overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle}}{x : ?^1[!^1[\mathsf{num}], \mathsf{num}] \vdash x(y,n).(\overline{y}\langle n\rangle \mid \overline{y}\langle n\rangle)}$$

- We need $\Gamma_1 = \Gamma_2 = \{y : !^\omega[\mathsf{num}], n : \mathsf{num}\}$ with defined $\Gamma_1(y) + \Gamma_2(y)$.
- Now this works, because $!^\omega[\mathsf{num}] + !^\omega[\mathsf{num}]$ is defined.
- Hence, $y : !^\omega[\mathsf{num}]$ can be in both Γ_1 and Γ_2.

Example: Summation Service



Recall the summation service(s) from the previous lecture:

$$!plusTwo(n, s).(r_1, r_2 : T)(\overline{plusOne}\langle n, r_1\rangle \mid r_1(y).\overline{plusOne}\langle y, r_2\rangle \mid r_2(z).\overline{s}\, z)$$

$$!plusTwo(n, s).(r : T)(\overline{plusOne}\langle n, r\rangle \mid r(y).\overline{plusOne}\langle y, s\rangle)$$

We have $T = \updownarrow^1[\text{num}]$ and the typing assignments are valid:

$n : \text{num} \quad s : !^1[\text{num}] \quad plusTwo : ?^\omega[\text{num}, ?^1[\text{num}]] \quad plusOne : ?^\omega[\text{num}, ?^1[\text{num}]]$

Properties of Typed Processes



1. Subject congruence (typing wrt structural congruence)
2. Type soundness (typing wrt reduction)
3. Partial confluence (influence of linearity on behavior)

Structural Congruence



The least congruence over processes that includes α-conversion of bound names and that is induced by the following four axioms:

$$P \mid Q \equiv Q \mid P$$

$$(P \mid Q) \mid R \equiv P \mid (Q \mid R)$$

$$(x : T)(P \mid Q) \equiv P \mid (x : T)Q \qquad \text{if } x \notin \mathsf{fn}(P)$$

$$(x : T)(y : U)P \equiv (y : U)(x : T)P \qquad x \neq y$$

Properties of Typed Processes



Theorem (Subject congruence)

If $P \equiv Q$ *then* $\Gamma \vdash P$ *if and only if* $\Gamma \vdash Q$.

Proof.

By induction on the proof of $P \equiv Q$.

- The base cases consist of the four axioms (plus reflexivity).
- The induction steps arise from the definition of structural congruence as a congruence relation. □

Properties of Typed Processes



- Typically, we would also like to have a property such as

$$\text{If } \Gamma \vdash P \text{ and } P \longmapsto Q \text{ then } \Gamma \vdash Q$$

  This is called preservation, type soundness, or subject reduction. It ensures that the (static) type discipline is preserved by the (dynamic) semantics.

Properties of Typed Processes



- Typically, we would also like to have a property such as

$$\text{If } \Gamma \vdash P \text{ and } P \longmapsto Q \text{ then } \Gamma \vdash Q$$

  This is called preservation, type soundness, or subject reduction. It ensures that the (static) type discipline is preserved by the (dynamic) semantics.

Properties of Typed Processes



- Typically, we would also like to have a property such as

$$\text{If } \Gamma \vdash P \text{ and } P \longmapsto Q \text{ then } \Gamma \vdash Q$$

  This is called preservation, type soundness, or subject reduction. It ensures that the (static) type discipline is preserved by the (dynamic) semantics.

- With linear resources, what should be the right property?
  Expectations:
  - We would like to see linear resources being consumed after reduction.
  - Also, we would like unlimited resources to be preserved.
  - Since reduction involves 'whole' capabilities, we focus on types $\updownarrow^m [\tilde{T}]$.

Properties of Typed Processes



- Typically, we would also like to have a property such as

$$\text{If } \Gamma \vdash P \text{ and } P \longmapsto Q \text{ then } \Gamma \vdash Q$$

  This is called preservation, type soundness, or subject reduction. It ensures that the (static) type discipline is preserved by the (dynamic) semantics.

- With linear resources, what should be the right property?
  Expectations:
  - We would like to see linear resources being consumed after reduction.
  - Also, we would like unlimited resources to be preserved.
  - Since reduction involves 'whole' capabilities, we focus on types $\updownarrow^m [\tilde{T}]$.

- To formalize this, we have a form of labeled reduction.
  A reduction relation instrumented with type information.

Labeled Reduction

$$P \xrightarrow{\alpha} Q$$

- Reduction captures an atomic communication step, as usual. Not an LTS!

Labeled Reduction

$$P \xrightarrow{\alpha} Q$$

- Reduction captures an atomic communication step, as usual. Not an LTS!
- The label α is a name (where the communication occurred) or m (indicating that the communication occurred on a local channel with multiplicity m).

Labeled Reduction

$$P \xrightarrow{\alpha} Q$$

- Reduction captures an atomic communication step, as usual. Not an LTS!
- The label α is a name (where the communication occurred) or m (indicating that the communication occurred on a local channel with multiplicity m).
- Type annotations may be changed by reduction. Example:

$$(x :\updownarrow^1 [])(\overline{x}\langle\rangle \mid x().P) \xrightarrow{1} (x : \emptyset^1[])P$$

The change of type annotation captures the fact that the linear channel x can no longer be used for communication.

Reduction Rules

R-Com

$$\overline{x}\langle\tilde{y}\rangle \mid x(\tilde{z}).P \xrightarrow{x} P[\tilde{y}/\tilde{z}]$$

R-RCom

$$\overline{x}\langle\tilde{y}\rangle \mid !x(\tilde{z}).P \xrightarrow{x} P[\tilde{y}/\tilde{z}] \mid !x(\tilde{z}).P$$

R-New1

$$\frac{P \xrightarrow{x} Q}{(x :\updownarrow^{\omega} [\tilde{T}])P \xrightarrow{\omega} (x :\updownarrow^{\omega} [\tilde{T}])Q}$$

R-New2

$$\frac{P \xrightarrow{x} Q}{(x :\updownarrow^{1} [\tilde{T}])P \xrightarrow{1} (x : \emptyset^{1}[\tilde{T}])Q}$$

R-New3

$$\frac{P \xrightarrow{\alpha} Q \qquad x \neq \alpha}{(x : T)P \xrightarrow{\alpha} (x : T)Q}$$

R-Par

$$\frac{P \xrightarrow{\alpha} Q}{P \mid R \xrightarrow{\alpha} Q \mid R}$$

R-Cong

$$\frac{P \equiv P' \qquad P' \xrightarrow{\alpha} Q' \qquad Q' \equiv Q}{P \xrightarrow{\alpha} Q}$$

R-IfT

$$\mathtt{if\ tt\ then}\ P\ \mathtt{else}\ Q \xrightarrow{1} P$$

R-IfF

$$\mathtt{if\ ff\ then}\ P\ \mathtt{else}\ Q \xrightarrow{1} Q$$

Type Soundness

$\Gamma - \alpha$ denotes the remaining capabilities in Γ after a communication on α:

$$\Gamma - \alpha \triangleq \begin{cases} \Delta, x : \emptyset^1[\tilde{T}] & \text{if } \alpha = x \text{ and } \Gamma = \Delta, x :\updownarrow^1 [\tilde{T}] \\ \Delta, x :\updownarrow^\omega [\tilde{T}] & \text{if } \alpha = x \text{ and } \Gamma = \Delta, x :\updownarrow^\omega [\tilde{T}] \\ \Gamma & \text{if } \alpha = 1 \text{ or } \alpha = \omega \end{cases}$$

Theorem

If $\Gamma \vdash P$ *and* $P \xrightarrow{\alpha} Q$ *then* $\Gamma - \alpha \vdash Q$.

Proof.

By induction on the derivation of $P \xrightarrow{\alpha} Q$. □

Partial Confluence



Theorem (Linear Communication is Deterministic)

If $\Gamma, x :\updownarrow^1 [\tilde{T}] \vdash P$, $P \xrightarrow{x} Q$, *and* $P \xrightarrow{x} R$ *then* $Q \equiv R$.

Proof.

By case analysis on the shape of P, together with uniqueness properties / normal forms derived from linear typing. □

Partial Confluence



Moreover, linear reduction is confluent with respect to all other reductions.

Define $\Gamma(\alpha)$ as follows:

$$\Gamma(\alpha) \triangleq \begin{cases} 1 & \text{if } \alpha = 1 \\ \omega & \text{if } \alpha = \omega \\ m & \text{if } \alpha = x \text{ and } \Gamma = \Delta, x : p^m[\tilde{T}] \end{cases}$$

Partial Confluence



Moreover, linear reduction is confluent with respect to all other reductions.

Define $\Gamma(\alpha)$ as follows:

$$\Gamma(\alpha) \triangleq \begin{cases} 1 & \text{if } \alpha = 1 \\ \omega & \text{if } \alpha = \omega \\ m & \text{if } \alpha = x \text{ and } \Gamma = \Delta, x : p^m[\tilde{T}] \end{cases}$$

Theorem (Partial Confluence)

If $\Gamma \vdash P$, $P \xrightarrow{\alpha} Q$, *and* $\Gamma(\alpha) = 1$ *then, for any* R *and* β *such that* $P \xrightarrow{\beta} R$, *either* $R \equiv Q$ *or* $R \xrightarrow{\alpha} S$ *and* $Q \xrightarrow{\beta} S$, *for some* S.

Informally: if P contains two possible communications, one of which is on a linear channel, then they may be performed in either order to reach the same result.

The Limits of Linear Typing



Consider the following process

$$P \triangleq x(z).\overline{y}\langle \mathtt{tt} \rangle \mid y(w).\overline{x}\langle \mathtt{ff} \rangle$$

The Limits of Linear Typing



Consider the following process

$$P \triangleq x(z).\overline{y}\langle \mathtt{tt} \rangle \mid y(w).\overline{x}\langle \mathtt{ff} \rangle$$

- P cannot reduce: on the left, the output on y is blocked by the input on x; on the right, the output on x is blocked by the input on y.

The Limits of Linear Typing



Consider the following process

$$P \triangleq x(z).\overline{y}\langle \mathtt{tt} \rangle \mid y(w).\overline{x}\langle \mathtt{ff} \rangle$$

- P cannot reduce: on the left, the output on y is blocked by the input on x; on the right, the output on x is blocked by the input on y.
- Still, the following judgment is derivable:

$$\mathtt{tt} : \mathsf{bool}, \mathtt{ff} : \mathsf{bool} \vdash (x :\updownarrow^1 [\mathsf{bool}])(y :\updownarrow^1 [\mathsf{bool}])P$$

The Limits of Linear Typing



Consider the following process

$$P \triangleq x(z).\overline{y}\langle \mathtt{tt} \rangle \mid y(w).\overline{x}\langle \mathtt{ff} \rangle$$

- P cannot reduce: on the left, the output on y is blocked by the input on x; on the right, the output on x is blocked by the input on y.
- Still, the following judgment is derivable:

$$\mathtt{tt} : \mathsf{bool}, \mathtt{ff} : \mathsf{bool} \vdash (x :\updownarrow^1 [\mathsf{bool}])(y :\updownarrow^1 [\mathsf{bool}])P$$

- Linear typing is not sufficient to exclude this kind of deadlocks.
  More advanced techniques are needed—a challenging research topic!

Summary



We closed our treatment of concurrency (and the course!) by considering:

- Strong bisimilarity in the π-calculus, in both early and late variants.
  When a name is received/substituted matters!
- A linear type system for the π-calculus. A resource-oriented view on communications.
  Two dimensions: multiplicities (how many times a name can be used) and capabilities/polarities (how a name can be used).
- Properties of linearly typed processes.



The End